

ConciliaEx

Plataforma para Familias con Personas Dependientes

MANUAL TÉCNICO

Versión 1.0

Proyecto Intermodular DAW

IES Albarregas · Mérida, Extremadura

José María Gutiérrez Barrena · 2DAW-B · 2025-2026

1. Introducción y Alcance

ConciliaEx es una aplicación web desarrollada con React, TypeScript y Vite, que conecta familias con personas dependientes con entidades especializadas de Extremadura.

1.1 Objetivos técnicos

- Arquitectura desacoplada Frontend / BaaS (Supabase)
- Autenticación segura con JWT gestionado por Supabase Auth
- Gestión de estado global ligera mediante Zustand con persistencia
- Navegación client-side con React Router v7
- Diseño accesible y responsive con Tailwind CSS v4
- TypeScript estricto (noUnusedLocals, verbatimModuleSyntax)

1.2 Stack tecnológico

Tecnología	Versión	Rol
React	19.2.5	UI Framework principal
TypeScript	6.0.3	Tipado estático
Vite	8.0.9	Bundler y servidor de desarrollo
Tailwind CSS	4.2.2	Estilos utility-first
Supabase JS	2.103.3	BaaS: Auth, DB y Storage
React Router DOM	7.14.1	Enrutamiento client-side
Zustand	5.0.12	Estado global con persistencia
shadcn/ui (Radix Nova)	—	Componentes accesibles
Lucide React	1.8.0	Iconografía SVG
browser-image-compression	2.0.2	Compresión de imágenes en cliente

2. Arquitectura de la Aplicación

2.1 Diagrama de capas

La aplicación sigue una arquitectura de tres capas lógicas claramente separadas:



2.2 Estructura de directorios



```
■■■ interfaces/      # Tipos TypeScript del dominio
■■■ lib/             # Constantes, estilos y validaciones
■■■ pages/           # Páginas por ruta
■   ■■■ admin/       # Panel de administración
■   ■■■ entidad/     # Dashboard entidad
■   ■■■ familia/     # Dashboard familia + solicitar
■   ■■■ legal/       # Páginas legales
■■■ store/           # Stores Zustand
```

3. Capa de Datos — Repository Pattern

Se aplica el patrón Repository para desacoplar la lógica de negocio del proveedor de datos. Las interfaces definen los contratos y las implementaciones concretas contienen el código específico de Supabase.

3.1 Repositorios disponibles

Interfaz	Implementación	Responsabilidad
AuthRepository	SupabaseAuthRepository	Registro, login, logout, sesión actual
ServicioRepository	SupabaseServicioRepository	CRUD servicios, búsqueda filtrada, visitas
SolicitudRepository	SupabaseSolicitudRepository	Creación y respuesta de solicitudes
PerfilRepository	SupabasePerfilRepository	Lectura/actualización de perfil y avatar
AdminRepository	SupabaseAdminRepository	Gestión de usuarios y servicios como admin

3.2 Cliente Supabase

```
// src/database/supabase/Client.ts

import { createClient } from '@supabase/supabase-js';

export const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL,
  import.meta.env.VITE_SUPABASE_ANON_KEY,
);
```

3.3 Esquema de base de datos

Tabla	Columnas clave	Descripción
perfiles	id, email, rol, nombre, nombre_entidad, activo	Envía a auth.users con datos de perfil por rol
servicios	id, entidad_id, nombre, tipo, ubicacion, telefono, estado, activo	Catálogo de servicios publicados por entidades
solicitudes	id, servicio_id, familia_id, entidad_id, urgente, estado	Solicitudes de plaza enviadas por familias

RLS habilitado en todas las tablas. Familias solo ven sus solicitudes; entidades acceden a las de sus servicios; admin usa la función RPC `admin_delete_user`.

4. Autenticación y Roles

4.1 Flujo de autenticación

1. El usuario rellena el formulario de registro/login.
2. El repositorio llama a `supabase.auth.signUp()` o `signInWithPassword()`.
3. Supabase devuelve un session con `access_token` y `refresh_token`.
4. El `authStore` persiste el objeto `Usuario` en `localStorage` (clave `conciliaex-auth`).
5. `onAuthStateChange` mantiene la sesión sincronizada entre pestañas.
6. En cada carga, `inicializar()` verifica la sesión y que el perfil esté activo.

4.2 Roles y permisos

Rol	Descripción	Rutas accesibles
familia	Usuario final que busca servicios	/familia/dashboard, /solicitar/:id, /perfil
entidad	Organización que publica servicios	/entidad/dashboard, /perfil
admin	Gestor de la plataforma	/admin, /perfil

4.3 Guards de ruta

- **ProtectedRoute:** Redirige a `/login` si no hay sesión activa.
- **RoleRoute:** Verifica que el rol del usuario coincida con el requerido.
- **AuthGuard:** Redirige al dashboard si el usuario ya tiene sesión.

5. Gestión de Estado — Zustand

Store	Persistencia	Responsabilidad principal
authStore	localStorage (usuario)	Sesión, registro, login, logout, inicialización
servicioStore	No	Catálogo, mis servicios, CRUD, búsqueda filtrada
solicitudStore	No	Crear, listar y responder solicitudes
perfilStore	No	Perfil del usuario autenticado, avatar
adminStore	No	Usuarios y servicios para el panel de admin
themeStore	localStorage (theme)	Modo claro/oscuro
langStore	localStorage (lang)	Idioma (es/en)

5.1 Prevención de race conditions

```
let searchIdActual = 0;

buscarServicios: async (filtros) => {

  const miId = ++searchIdActual;

  set({ cargandoBusqueda: true });

  const resultados = await servicioRepo.buscarServicios(filtros);

  if (miId !== searchIdActual) return; // respuesta obsoleta

  set({ resultados });

},
```

5.2 Reset al cerrar sesión

La función `cerrarSesion()` llama a `limpiarStores()`, que resetea todos los stores con datos de usuario para evitar que datos sensibles persistan entre sesiones.

6. Componentes Reutilizables

6.1 Componentes de Dashboard

Componente	Props principales	Descripción
KpiCard	label, value, color, highlight	Tarjeta de métrica con valor destacado
DataTable<T>	columns, data, keyExtractor	Tabla tipada con columnas configurables
TabBar<T>	tabs, active, onChange	Barra de pestañas genérica con badges
Modal	onClose, children, maxWidth	Modal con overlay y propagación detenida
EmptyState	title, subtitle, action	Estado vacío con CTA opcional
StatusBadge	activo, labelActivo, labelInactivo	Badge de estado activo/inactivo
ToggleActivoButton	activo, labelActivar, onClick	Botón toggle de activación
Skeleton (varios)	—	Placeholders animados por tipo de contenido

7. Hooks Personalizados

Hook	Descripción
useServicioForm()	Gestiona formularios de creación/edición de servicios con compresión de imagen.
useRespuestaSolicitud()	Controla el modal de respuesta a solicitudes en el dashboard de entidad.
useSolicitudesFiltro()	Filtra solicitudes por estado y texto de búsqueda de forma reactiva.
useAdminFiltros()	Filtros combinados (búsqueda, rol, estado) para el panel de admin.
useT()	Devuelve las traducciones del idioma activo en langStore.

8. Internacionalización (i18n)

- **src/i18n/es.ts**: Interfaz Translations y objeto con todas las cadenas en español (fuente de verdad).
- **src/i18n/en.ts**: Traducciones al inglés, tipadas contra la misma interfaz para garantizar la paridad.
- **src/i18n/useT.ts**: Hook que lee langStore y retorna el objeto de traducciones correcto.
- **langStore**: Persiste la preferencia en localStorage bajo la clave conciliaex-lang.

9. Sistema de Estilos — Tailwind CSS v4

9.1 Configuración

Tailwind v4 se integra mediante el plugin oficial `@tailwindcss/vite`, sin fichero PostCSS separado. El tema completo se define en `src/index.css` mediante variables CSS:

```
@import 'tailwindcss';

@import 'tw-animate-css';

@custom-variant dark (&:is(.dark *));

:root {

  --primary:    #0d9488;

  --background: #faf7f4;

  --foreground: #1c1412;

}

.dark {

  --background: #1c1917;

  --primary:    #2dd4bf;

}
```

9.2 Modo oscuro

La clase `.dark` se aplica al elemento `html` mediante `themeStore`. La preferencia se persiste en `localStorage`. El cambio de tema es instantáneo sin recargar la página.

10. Enrutamiento — React Router v7

```
/ (Landing)

/catalogo

/login          [AuthGuard - solo sin sesión]

/registro       [AuthGuard]

/registro/familia [AuthGuard]

/registro/entidad [AuthGuard]

/recuperar

/nueva-password

/legal/*

                [ProtectedRoute - requiere sesión]

/familia/dashboard [RoleRoute: familia]

/solicitar/:id     [RoleRoute: familia]

/entidad/dashboard [RoleRoute: entidad]

/admin            [RoleRoute: admin]

/perfil           [todos los roles]

* (404 NotFound)
```

11. Variables de Entorno

```
# .env (no commitear – incluido en .gitignore)

VITE_SUPABASE_URL=https://xxxxxxxxxxxxxxxxx.supabase.co

VITE_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

■ ■ Atención

La anon key de Supabase es segura en cliente: solo permite operaciones autorizadas por las políticas RLS. La service_role key NUNCA debe incluirse en el frontend.

12. Validaciones del Lado Cliente

Función	Validación
esTelefonoValido(tel)	9 dígitos, empieza por 6/7/8/9, acepta +34 y espacios/guiones
esEmailValido(email)	Formato estándar de email (regex básico)
esCifValido(cif)	Formato CIF español: letra + 7 dígitos + letra/dígito

13. Optimizaciones de Rendimiento

13.1 Compresión de imágenes

```
const OPTS = { maxSizeMB: 1, maxWidthOrHeight: 1920, useWebWorker: true };

const file = archivo.size > 1_048_576

  ? await imageCompression(archivo, OPTS)

  : archivo;
```

13.2 Debounce en búsqueda

El catálogo implementa debounce de 400 ms en los filtros para reducir llamadas a Supabase mientras el usuario escribe.

13.3 Skeletons de carga

Cada sección con datos asíncronos muestra skeletons animados (KpiCardSkeleton, ServicioCardSkeleton, SolicitudCardSkeleton, CatalogoCardSkeleton, TableSkeleton) durante la carga.

13.4 Code splitting

Vite genera automáticamente chunks separados por ruta durante el build de producción.

14. Convenciones de Desarrollo

14.1 Commits Git

Mensajes en español, sin prefijos de tipo. Ejemplos: "Inicio", "Setup Tailwind shadcn y router", "Dashboard familia con KPIs".

14.2 TypeScript

Configuración estricta: `noUnusedLocals`, `noUnusedParameters`, `erasableSyntaxOnly` y `verbatimModuleSyntax`.

14.3 Nomenclatura

- Componentes: PascalCase (DashboardFamilia, KpiCard)
- Hooks: camelCase con prefijo use (useServicioForm, useT)
- Stores: camelCase con sufijo Store (useAuthStore)
- Repositorios: PascalCase con prefijo Supabase (SupabaseAuthRepository)

15. Limitaciones y Puntos de Mejora

1. Notificaciones en tiempo real: Las solicitudes no notifican al panel de entidad en tiempo real. Supabase Realtime podría integrarse.
2. Tests automatizados: El proyecto no incluye tests unitarios ni de integración (Vitest + Testing Library recomendados).
3. Paginación del catálogo: Se cargan todos los servicios activos en una sola query. Con alto volumen habría que paginar.
4. Caché de datos: Los stores no implementan caché temporal; las peticiones siempre consultan Supabase.
5. Emails transaccionales: La recuperación de contraseña usa el email por defecto de Supabase. En producción se debería configurar SMTP personalizado.

— Fin del Manual Técnico —